

Σοφος – 前向安全可搜索加密

摘要

1. SSE (Searchable Symmetric Encryption) 目标：

- 在不可信服务器上存储加密数据库时，支持查询操作。
- 保证查询和数据的隐私，同时允许向服务器泄露一些受控信息。

2. 动态SSE的挑战：

- 最新研究表明，动态方案（数据可更新）会泄露关于更新关键词的信息。
- 这种泄露可能导致适应性攻击，破坏查询的隐私。

3. 解决方法：前向隐私 (Forward Privacy)：

- 为了防止攻击，需要设计前向隐私方案，确保更新操作不会泄露新插入的数据是否与之前的查询匹配。

4. Σοφος 方案：

- Σοφος 是一个具有前向隐私的SSE方案。
- 性能接近现有不太安全的方案，同时比之前的前向隐私方案更加简单和高效。
- 主要依赖陷门置换 (Trapdoor Permutations)，不使用类似 ORAM 的构造。
- 该方案在安全性与性能的权衡上达到了最佳平衡。

5. 实施与评估：

- 提供了 Σοφος 的实现和评估结果，证明其具有实际的高效性。

1 引言

1. 背景与挑战：

- 安全的云应用需要能够高效且私密地查询存储在不可信服务器上的加密数据库。
- 理想情况下，查询协议应向服务器泄露零信息。尽管可以使用多方计算、全同态加密或不可知内存等技术来实现，但这些技术因其通用性而不实用，速度较慢，甚至不如直接下载整个数据库进行本地搜索。

2. SSE方案的基本概念：

- SSE (Searchable Symmetric Encryption)** 是一种结构化加密技术，用于支持如搜索索引或搜索树等结构的加密查询。
- 通过在效率和泄露之间做权衡，SSE允许一些信息泄露，以便服务器能够轻松地在加密数据中查找匹配的项。
- 这种泄露通常来自对称加密方式，利用确定性加密来使服务器能发现加密token之间的匹配关系，无需运行昂贵的协议或计算。

3. SSE方案中的信息泄漏：

- 不同的SSE方案泄漏的信息量不同，从完全暴露关键词出现模式（如CipherCloud、skyhigh等方案）到仅泄漏查询相关的模式（基于反向索引的方案）。
- 即使是少量的泄漏也能被攻击者利用来推断用户的查询，从而引发泄漏滥用攻击。

4. 攻击与漏洞：

- 攻击者通过泄漏的小信息，可以推测出用户的查询；对于更大规模的泄露，攻击者甚至可以恢复加密数据库的明文。
- 在动态数据库场景中，攻击者能够通过向数据库中注入少量新文档来暴露旧查询的内容。该攻击被称为“适应性攻击”，可通过插入少于10个新文档来实现。
- 目前几乎所有的 SSE 方案都易受到这种攻击，因为服务器可以通过观察新插入的文档是否匹配以前的查询来推测信息。

5. 前向隐私 (Forward Privacy) 解决方案：

- 为防止此类攻击，需要设计**前向隐私 (Forward Privacy)** 的 SSE 方案，确保更新操作不会泄露新插入的元素是否与先前的查询匹配。
- 现有的前向隐私方案包括基于 ORAM (如TWO-RAM) 的方案，以及 Chang 和 Mitzenmacher 的方案，但这些方案在效率上存在问题。
- 前者需要大量的带宽和服务器存储，后者则依赖复杂的ORAM技术，导致更新操作带来较大的带宽开销。

贡献

1. $\Sigma\phi\phi\phi$ 方案概述：

- $\Sigma\phi\phi\phi$ 是一种前向隐私 SSE 方案，具有最佳的搜索和更新复杂度，既考虑计算复杂度，也考虑通信复杂度。
- 它结合了前向隐私构造的安全保证和具有更多泄露的构造的渐近效率。
- 该方案使用简单的加密工具（仅使用伪随机函数和陷门置换），无需依赖ORAM技术，且易于理解。

2. 安全性和扩展性：

- 提供了完整的安全性证明，针对诚实但好奇的对手，且可以轻松扩展到对抗恶意对手的安全性 ($\Sigma\phi\phi\phi - \epsilon$)，无需修改服务器端构造。

3. 实践中的效率：

- 提供了开源实现，展示了即使在持久存储上， $\Sigma\phi\phi\phi$ 在实践中对搜索和更新都非常高效。

4. 与前期工作比较：

- 通过与先前工作进行比较（如表1所示），证明了 $\Sigma\phi\phi\phi$ 在安全性和效率上的优势。

2 相关工作

1. SSE的起源与早期发展：

- **Song et al. (2000)**：首次提出SSE，解决方案的搜索时间与数据库大小成线性关系。
- **Chang和Mitzenmacher (2005)**：提出了一个具有前向隐私的SSE方案，尽管搜索时间依然是线性的。
- **Curtmola et al. (2006)**：首次正式考虑泄露问题，并设计了一个静态场景下的基于索引的 SSE方案，实现了亚线性搜索复杂度。

2. 动态和子线性方案的出现：

- **Kamara et al. (2012)**：提出了第一个动态且具有亚线性复杂度的SSE方案，但该方案泄露了更新文档中关键词的哈希值。
- **Kamara和Papamanthou (2013)**：改进了该方案，减少了泄露信息，但增加了服务器的空间复杂度，且仍不具备前向隐私。

3. 支持布尔查询和大规模数据库的SSE方案：

- **CJJ+13, CJJ+14**: 提出了支持布尔查询的索引基SSE方案，并对非常大的数据库和动态性进行了优化。
- **JJK+13, PKV+14**: 针对三方设置中的SSE进行了研究，部分特性也在其他工作(PKV+14)中得到研究。

4. 前向隐私和可验证SSE:

- **Stefanov et al. (2014)**: 首次明确提出了前向隐私的概念，并设计了一个动态、亚线性的前向隐私SSE方案。
- **Kurosawa和Ohtaki (2012)**: 研究了可验证的SSE方案，即能防御活跃对手的攻击。
- **Bost et al. (2016)**: 提出了动态且高效的前向隐私SSE方案。

5. 存储局部性和ORAM的挑战:

- **Cash和Tessaro (2014)**: 提出了搜索加密的存储局部性下界，证明无法同时实现最优的服务端存储和局部性。
- **Asharov et al. (2016)**: 证明了这个下界的紧性。
- **Chase和Kamara (2010)**: 将SSE扩展到任意结构化数据，如图、矩阵或标记数据。
- 最近出现了结构化加密的新应用，例如加密图上的近似最短距离查询[MKNK15]。
- **ORAM与SSE**: ORAM可用于构建SSE，但由于带宽开销大、客户端存储需求高，且需要多次往返，导致ORAM在SSE中的应用不现实。

6. 泄露研究与前向隐私的必要性:

- **Islam et al. (2012)**、**Cash et al. (2015)**、**Zhang et al. (2016)**: 研究了SSE中泄露的后果，并展示了前向隐私对于防止泄露滥用攻击的重要性。

3 预备知识

陷门置换

1. 陷门置换 (TDP) :

- **定义**: 一个置换 π 是一个在集合 D 上的映射，具有“陷门”特性，意味着：
 - 使用公开密钥 PK 可以轻松计算 π (即， $\pi(x)$ 可以计算出来)。
 - 但只有在知道私钥 SK 的情况下，才能高效地计算逆置换 π^{-1} (即，找到 x 使得 $\pi(x) = y$)。

2. 形式化定义:

- 对于给定的安全参数 λ ，陷门置换 π 的安全性定义为：对于每个对手 $\mathcal{A}$,

$$Adv_{\pi, \mathcal{A}}^{OW}(\lambda) \leq \text{negl}(\lambda)$$

◦

- 其中:

- $Adv_{\pi, \mathcal{A}}^{OW}(\lambda)$ 表示对手 $\mathcal{A}$ 通过挑战 π 的加密操作来猜测 y 的概率，即

$$Pr[y \stackrel{\$}{\leftarrow} M, (SK, PK) \leftarrow \text{KeyGen}(1^\lambda), x \leftarrow \mathcal{A}(1^\lambda, PK, y) : \pi_{PK}(x) = y]$$

- 这个概率表示对手 $\mathcal{A}$ 在不知道私钥 SK 的情况下，能否有效地从 y 和 PK 推导出 x ，使得 $\pi_{PK}(x) = y$ 。
- 如果 $\mathcal{A}$ 能以大于微不足道的概率找到 x 并满足 $\pi_{PK}(x) = y$ ，那么这个 TDP 就被破解了，意味着该 TDP 不是单向的。

- 如果这个概率 $Adv_{\pi, \mathcal{A}}^{OW}(\lambda)$ 非常小（即接近于0），那么说明 TDP 是单向的，攻击者无法有效反向计算。

3. 单向性和可逆性：

- 对于每个 $x \in \mathcal{D}$ ，TDP 满足：
 - $\pi_{PK}(\pi_{SK}^{-1}(x)) = x$ ，即如果先用私钥反向计算，再用公钥应用 π ，能得到原始输入 x 。
 - $\pi_{SK}^{-1}(\pi_{PK}(x)) = x$ ，即如果先用公钥应用 π ，再用私钥反向计算，能得到原始输入 x 。
- π_{PK} 和 π_{SK}^{-1} 都能在多项式时间内计算。

4. 迭代应用：

- 记 $\pi_{PK}^{(c)}(x)$ 为 $\pi_{PK}(x)$ 的迭代应用 c 次，即重复应用公钥 π 函数 c 次。
- 记 $\pi_{SK}^{(-c)}(x)$ 为 $\pi_{SK}^{-1}(x)$ 的迭代应用 c 次，即重复应用私钥 π 逆函数 c 次。

对称可搜索加密

1. 数据库表示：

- 数据库 **DB** 是一个由索引-关键词对组成的元组，表示为：
 $DB = (ind_i, W_i)_{i=1}^D$ ，其中：
 - ind_i 是文档的索引，属于集合 $\{0, 1\}^\ell$ （即，文档的唯一标识符，长度为 ℓ 位）。
 - W_i 是文档 ind_i 的关键词集合，属于 $\{0, 1\}^*$ （即每个文档的关键词是从所有可能的关键词中选出的）。
- **W** 表示数据库中所有关键词的集合：
 $W = \bigcup_{i=1}^D W_i$ ，其中 D 是文档的总数。
- **N** 是文档-关键词对的总数：
 $N = \sum_{i=1}^D |W_i|$ ，即数据库中所有文档和关键词的配对总数。
- **DB(w)** 表示包含关键词 w 的文档集合：
 $DB(w) = \{ind_i \mid w \in W_i\}$ ，即文档 ind_i 包含关键词 w 。

2. 动态可搜索加密 (DSE) 方案：

- 一个 DSE 方案 Π 包含一个 **Setup** 算法和两个协议：**Search** 和 **Update**，它们在客户端和服务端之间交互。

Setup(DB)：

- 输入：数据库 **DB**。
- 输出：一对 **(EDB, K, σ)**，其中：
 - **EDB** 是加密后的数据库。
 - **K** 是秘密密钥。
 - σ 是客户端的状态（可以用于后续操作）。

Search(K, q, σ ; EDB)：

- 输入：
 - 客户端输入：密钥 K 、状态 σ 、搜索查询 q （对于单关键词搜索， q 是一个特定的关键词 w ）。
 - 服务器输入：加密数据库 **EDB**。
- 输出：搜索的结果由两个部分组成：
 - $Search_C(K, q, \sigma)$ ：客户端的计算部分。

- $\text{Search}_S(\text{EDB})$: 服务器的计算部分。

Update(K, σ , op, in; EDB):

- 输入:
 - 客户端输入: 密钥K、状态 σ 、操作类型op (如“添加”或“删除”操作)、输入内容in (包含文档索引 ind 和关键词集合W)。
 - 服务器输入: 加密数据库 EDB。
- 输出: 更新的结果由两个部分组成:
 - $\text{Update}_C(K, \sigma, op, in)$: 客户端的更新计算部分。
 - $\text{Update}_S(\text{EDB})$: 服务器的更新计算部分。
- **操作类型**: 操作类型op可以是“add” (添加) 或“del” (删除), 表示向数据库中添加或删除文档-关键词对。

SSE的安全性

1. SSE的安全性属性:

- **正确性 (Correctness)** :
SSE方案的搜索协议对于每个查询必须返回正确的结果, 除了以忽略的概率外。
- **保密性 (Confidentiality)** :
SSE方案的保密性通过现实世界与理想世界的模型进行形式化。方案通过一个**泄露函数** $\mathcal{L} = (\mathcal{L}^{Stp}, \mathcal{L}^{Srch}, L^{Updt})$ 来描述协议泄露给对手的所有信息, 并通过状态算法来形式化。保密性要求方案泄露的信息不超过通过泄露函数能推导出的内容。

SSE方案的保密性定义:

- 使用**SSEReal**和**SSEIdeal**两个游戏来定义方案的保密性。
- 对手A选择一个数据库DB, 并在真实世界 (通过 $\text{Setup}(\text{DB})$ 生成加密数据库EDB) 或理想世界 (通过模拟器S生成的 $S(\mathcal{L}^{Stp}(\text{DB}))$) 中获得数据库。
- 然后, A可以进行搜索和更新查询, 分别通过**Search(q)**和**Update(op, in)**协议进行。在真实游戏中, 这些查询会泄露搜索和更新的实际信息; 而在理想游戏中, 模拟器S仅会泄露由泄露函数 (如 $\mathcal{L}^{Srch}$ 和 L^{Updt}) 生成的消息。
- 如果A不能通过这些信息区分真实游戏与理想游戏的执行结果, 且该区分的概率是微不足道的 (即negligible), 则我们说该方案是**L-适应安全** (L-adaptively-secure)。

2. 常见泄露 (Common Leakage) :

- **搜索模式 (Search Pattern)** :
很多SSE方案会泄露客户端向服务器发送的令牌的重复信息, 因此泄露出查询关键词的重复性。这种泄露被称为**搜索模式**, 通常发生在静态方案中, 指的是查询关键词的重复。
- **查询模式 (Query Pattern)** :
如果更新操作的关键词也会泄露信息 (即更新时的关键词重复性), 我们称之为**查询模式**。这发生在动态方案中, 表示查询和更新的关键词重复性。

3. 泄露函数L的定义:

- **查询列表Q**: 泄露函数L将维护一个查询列表Q, 记录所有已执行的查询。每个查询由一个时间戳i和操作类型组成, 对于搜索查询是(i, w), **对于更新查询是(i, op, in)**, 其中op是操作类型, in是更新的输入。
 - $sp(x) = \{j : (j, x) \in Q\}$ (only matches search queries)
 - 表示搜索模式: 定义为所有与关键词x匹配的查询。

- $qp(x) = \{j : (j, x) \in Q \text{ or } (j, \text{op}, \text{in}) \in Q \text{ and } x \text{ appears in in}\}$
 - 表示查询模式：定义为所有与关键词x匹配的查询或更新操作，其中x出现在更新输入in中。

4. 历史记录 (History) :

- **HistDB(w)**: 表示关键词w的历史记录，即与w匹配的所有文档的历史添加记录，包括已添加后删除的文档，以及重复添加的文档，按照添加顺序排列。
- **Hist(w)**: 表示关键词w的修改历史。它由两个部分组成：
 - $\text{DB}_0(w)$: 关键词w在初始化时匹配的文档集合。
 - **UpHist(w)**: 关键词w的更新历史，记录了所有与w相关的更新操作（如添加或删除文档）。

例如，假设有两个文档 D_{ind_1} 和 D_{ind_2} 匹配关键词w:

- D_{ind_1} 在第2次更新时添加。
- D_{ind_2} 在第5次更新时添加。
- D_{ind_1} 在第14次更新时被删除。
- 则 $\text{UpHist}(w) = [(2, \text{add}, D_{ind_1}), (5, \text{add}, D_{ind_2}), (14, \text{del}, D_{ind_1})]$ 。

4 前向安全

4.1 定义

1. Forward Privacy:

Forward privacy是动态SSE方案中的一个强安全属性，确保更新操作不会泄露任何关于更新关键词的信息。具体来说，更新后的文档不能让服务器推断出它匹配了之前查询过的关键词。换句话说，更新操作的执行不会暴露与已查询关键词的关系。

2. 正式定义:

一个 $\mathcal{L}$ -适应安全的SSE方案 Σ ，如果其更新泄露函数 $\mathcal{L}^{\text{Updt}}$ 可以表示为:

$$\mathcal{L}^{\text{Updt}}(\text{op}, \text{in}) = \mathcal{L}'(\text{op}, \{(\text{ind}_i, \mu_i)\})$$

其中， $\{(\text{ind}_i, \mu_i)\}$ 是一个包含修改文档 ind_i 及其对应修改关键词数量 μ_i 的集合。该定义扩展了Stefanov等人 ([SPS14]) 对forward privacy的定义，后者只关注新增文档的情况，而这里的定义包括了**更新文档**，即既包括添加文档也包括删除文档。

3. 与 Backward Privacy 的关系:

Stefanov等人还讨论了 **Backward Privacy**，它确保删除的文档不能被查询。支持同时具备 **Forward** 和 **Backward Privacy** 的唯一方案是基于 **Oblivious RAM (ORAM)**，但这些方案通常效率较低。

4. 效率问题:

虽然已经有针对 **forward privacy** 的高效方案（例如[CM05, SPS14]），但这些方案常常面临带宽或存储消耗过大的问题。

4.2 前向隐私的必要性

1. 泄漏攻击:

Islam等人[IKK12]和Cash等人[CGPR15]研究表明，即使是微小的泄漏也能被攻击者利用来推测客户端的查询。他们的研究涵盖了静态和动态数据库，攻击者可以通过注入新文档来破坏查询隐私。

2. Zhang等人的改进:

Zhang等人[ZKP16]改进了文件注入攻击, 描述了非自适应攻击和自适应攻击两种类型。**自适应攻击**对非前向隐私的方案特别有效。该攻击通过提交少量新文档 ($\log 2T$ 或 $W/T + \log T$ 文档, 取决于攻击者对数据库的了解) 来揭示先前查询过的关键词。例如, 在 $T=200$ 的情况下, 攻击者只需提交不到10个新文档就能突破查询隐私。

3. 前向隐私的重要性:

前向隐私不仅提高了安全性, 还在功能上带来效率提升。它允许**在线构建**加密数据库, 而不像其他SSE方案需要时间和空间消耗的索引构建 (例如, 创建倒排索引)。

4.3 由前向隐私引起的约束

1. 存储问题:

- 如果攻击者先发起一个搜索查询, 然后提交删除操作, 对于加密数据库中的存储位置没有修改 (即**现有方案大多数不修改数据库**), 且该方案有空间回收机制, 那么攻击者可以通过观察数据库的变化, 推测出该关键词最近被搜索过。
- 为了保持前向隐私, 方案必须保证加密对的物理位置在查询前后完全无关, 这使得存储方案的安全性接近于**可调整大小的ORAM**, 但这会导致效率下降。

2. 局部性问题:

- 内存访问局部性问题**: Cash和Tessaro (2014) 研究发现, 要实现最小的内存局部性, 必须增加加密数据库的大小, 无法在不增加大小的情况下优化局部性。
- 局部性与前向隐私的矛盾**: 对于动态SSE方案, 局部性和前向隐私是**不可调和**的概念。前向隐私要求更新后的密文位置与原位置无关, 导致无法优化内存局部性。
- 局部性优化的代价**: 如果每次搜索时都更新加密数据库, 局部性优化带来的效率提升将被重写成本所抵消。
- 方案选择**: 要实现前向隐私和内存局部性, 必须对加密数据库进行较大范围的修改, 不论是在搜索还是更新过程中。

5 Σοφος 构造

5.1 一般思想

- 倒排索引方案**: 在倒排索引方案中, 每个关键词 w 都会维护一个文档索引列表 $\{\text{ind}_0, \dots, \text{ind}_{n_w}\}$, 其中每个文档索引会被加密, 并存储在由 w 和索引 c 决定的逻辑位置 $UT_c(w)$ 中。
- 更新操作**: 当客户端想要添加一个匹配关键词 w 的文档时, 他会计算一个新的位置 $UT_{n_w+1}(w)$, 加密文档索引 e , 并将 $(UT_{n_w+1}(w), e)$ 发送到服务器。
- 查询操作**: 当客户端进行查询时, 会生成一个搜索令牌 $ST(w)$, 这个令牌将允许服务器重新计算更新令牌的位置。**关键点**是, 更新令牌应该是不可追踪的, 直到客户端发出相应的查询令牌。
- 查询令牌的不可追踪性**: 搜索令牌 $ST_c(w)$ 是由客户端生成的, 并且应该对未来的更新令牌 $UT_i(w)$ 保持不可追踪性, 特别是, 服务器无法通过 $ST_c(w)$ 推导出 $ST_i(w)$ (对于 $i > c$ 的未来令牌)。
- 令牌生成的解决方案**:
 - 常规方案**: 如果客户端为每个查询都生成 $O(n_w)$ 令牌并发送到服务器, 可能会导致性能问题, 特别是在资源受限的设备上。

为啥?

- **新方案**：使用**陷门置换 (trapdoor permutation)**。服务器可以通过一个公钥从 $ST_i(w)$ 计算令牌 $ST_{i+1}(w)$ ，但是只有客户端才能从 $ST_i(w)$ 生成 $ST_{i+1}(w)$ ，而服务器则可以通过公钥从 $ST_c(w)$ 计算出所有先前的令牌 $ST_i(w)$ (对于 $0 \leq i < c$)。

6. 安全性保证：

- 更新令牌通过带有密钥的哈希函数 (H) 从查询令牌派生出来。关键点是哈希函数 H 必须是随机神谕，这对于方案的安全性至关重要。

7. 随机预言机模型：方案的安全性会在假设哈希函数 H 是一个随机神谕的模型下进行证明。

关键概念：

- **更新令牌 (Update Token)**：用于存储每个文档在加密数据库中的位置。
- **搜索令牌 (Search Token)**：用于进行搜索查询，并生成相应的更新令牌。
- **陷门置换 (Trapdoor Permutation)**：一种加密方法，其中只有客户端可以生成某些令牌，而服务器只能计算已有的令牌。
- **哈希函数 (Hash Function)**：用于从查询令牌生成更新令牌，并确保哈希函数的**抗预像性**来保证安全。

抗预像性 要求对于给定的输出哈希值 h ，逆向找到输入消息 m 变得极其困难，并且计算复杂度应该足够高，直到无法在实际时间内完成。

基本结构

1. 基本思想：该方案的设计参考了前面提到的思路，尤其是在令牌生成方面。主要操作是**插入新的关键词/文档对**，而不支持其他类型的更新。

2. 组件说明：

- π ：陷门置换函数 (trapdoor permutation)。
- F ：伪随机函数 (PRF)。
- H_1 和 H_2 ：有密钥的哈希函数，分别生成长度为 μ 和 ℓ 的输出。

3. 客户端操作：

- **W**：一个映射，每个插入的关键词 w 对应于当前的搜索令牌 $ST_c(w)$ 和计数器 $c = n_w - 1$ 。

n_w 是关键字 w 的搜索结果集的大小， a_w 是查询的关键字 w 历史上添加到数据库的次数。

◦ 文档插入时：

- 如果文档与关键词 w 匹配，客户端会更新 $W[w]$ ：生成新的搜索令牌 $ST_{c+1}(w) = \pi^{-1}(ST_c(w))$ 。
- 如果文档未匹配该关键词，则选择一个新的随机令牌 $ST_0(w)$ 并存入 W 。

- **位置**：通过密钥哈希函数从搜索令牌衍生出位置更新令牌。

4. 加密过程： $\Sigma\phi\phi\phi\phi - B$ 的初始化过程不需要数据库输入，支持在线执行加密而不影响前向隐私。

$\Sigma\phi\phi\phi\phi - B$

该方案被定义为以下三个算法：

1. Setup()

生成一个对称密钥 K_s 和一对公私钥 (SK, PK)。并初始化两个映射 $\mathbf{W}$ 和 $\mathbf{T}$ ，其中， $\mathbf{W}$ 用于存储客户端中每个关键词的最新的搜索令牌， $\mathbf{T}$ 用于存储服务器端的加密项。

2. Update(add, w , ind, σ ; EDB)

客户端通过映射表 $\mathbf{W}$ 找到对应关键词的搜索令牌 $ST_c(w)$ 和计数器 c ，用私钥陷门置换计算新的搜索令牌 $ST_{c+1}(w)$ 并用其更新映射表 $\mathbf{W}$ 。接下来客户端通过对称密钥 K_s 和关键词 w 生成密钥 K_w ，与新的搜索令牌 $ST_{c+1}(w)$ 分别用两个不同的哈希函数计算，一个结果作为新的更新令牌 $UT_{c+1}(w)$ ，另一个结果与要更新的文档 id 异或后得到加密值 e 。最终将这两个结果发送给服务器。

服务器收到 $UT_{c+1}(w)$ 和 e 后，更新 $\mathbf{T}[UT_{c+1}] = e$ 。一次更新结束。

3. Search(w , σ ; EDB)

客户端先计算生成关键词 w 的密钥 K_w ，然后和映射表 $\mathbf{W}$ 中对应关键词的搜索令牌 $ST_c(w)$ 和计数器 c 一起发送给服务器。

服务器收到后，从 $i = c$ 开始，用哈希函数 H_1 计算 K_w 和搜索令牌 $ST_i(w)$ 得到搜索令牌 $UT_i(w)$ ，从 $\mathbf{T}$ 中取得对应的加密项 e ，然后与 $H_2(K_w, ST_i)$ 异或得到文档 id。接着用公钥陷门置换计算前一个搜索令牌 ST_{i-1} ，重复上述步骤直到取得所有文档 id。

Algorithm 1 Σοφος-B: Forward private SSE scheme with large client storage.

Setup()

```
1:  $K_S \xleftarrow{\$} \{0, 1\}^\lambda$ 
2:  $(SK, PK) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $\mathbf{W}, \mathbf{T} \leftarrow \text{empty map}$ 
4: return  $((\mathbf{T}, PK), (K_S, SK), \mathbf{W})$ 
```

Search(w , σ ; EDB)

```
Client:
1:  $K_w \leftarrow F_{K_S}(w)$ 
2:  $(ST_c, c) \leftarrow \mathbf{W}[w]$ 
3: if  $(ST_c, c) = \perp$ 
4:   return  $\emptyset$ 
5: Send  $(K_w, ST_c, c)$  to the server.
Server:
6: for  $i = c$  to 0 do
7:    $UT_i \leftarrow H_1(K_w, ST_i)$ 
8:    $e \leftarrow \mathbf{T}[UT_i]$ 
9:    $\text{ind} \leftarrow e \oplus H_2(K_w, ST_i)$ 
10:  Output each ind
11:    $ST_{i-1} \leftarrow \pi_{PK}(ST_i)$ 
12: end for
```

Update(add, w , ind, σ ; EDB)

```
Client:
1:  $K_w \leftarrow F(K_S, w)$ 
2:  $(ST_c, c) \leftarrow \mathbf{W}[w]$ 
3: if  $(ST_c, c) = \perp$  then
4:    $ST_0 \xleftarrow{\$} \mathcal{M}$ ,  $c \leftarrow -1$ 
5: else
6:    $ST_{c+1} \leftarrow \pi_{SK}^{-1}(ST_c)$ 
7: end if
8:  $\mathbf{W}[w] \leftarrow (ST_{c+1}, c + 1)$ 
9:  $UT_{c+1} \leftarrow H_1(K_w, ST_{c+1})$ 
10:  $e \leftarrow \text{ind} \oplus H_2(K_w, ST_{c+1})$ 
11: Send  $(UT_{c+1}, e)$  to the server.
Server:
12:  $\mathbf{T}[UT_{c+1}] \leftarrow e$ 
```

AI 总结

1. Setup (初始化)

1. 生成密钥:

- 生成一个对称密钥 K_S ，用于伪随机函数 (PRF) F_{K_S} 。
- 生成一对公私钥 (SK, PK)，用于陷门置换 (trapdoor permutation) π 。

2. 初始化数据结构:

- W: 客户端上的一个映射，记录每个关键字的最新搜索令牌 $ST_c(w)$ 和计数器 c 。
- T: 服务器上的一个映射，记录加密数据的位置及其值。

3. 返回密钥和初始化结构:

这一步完成了加密系统的初始化。

2. Search (搜索)

工作流程：

- 客户端部分：

1. 使用 PRF F_{K_S} 生成关键字 w 的密钥 K_w 。
2. 从映射 W 中获取最新的搜索令牌 $ST_c(w)$ 和计数器 c (即关键字匹配文档的数目 $n_w - 1$)。
如果 $ST_c(w)$ 不存在, 返回空结果。
3. 将 $(K_w, ST_c(w))$ 发送给服务器。

- 服务器部分：

1. 根据接收到的搜索令牌 $ST_c(w)$, 从 c 开始迭代, 计算更新令牌 UT_i :
$$UT_i = H_1(K_w, ST_i)$$
2. 用 UT_i 从服务器的存储映射 T 中提取密文 e 。
通过异或操作解密 $ind = e \oplus H_2(K_w, ST_i)$ 获得文档索引 ind 。
3. 依次输出每个匹配文档的索引 ind , 并使用公钥陷门置换 π 计算前一个搜索令牌 ST_{i-1} :
$$ST_{i-1} = \pi_{PK}(ST_i)$$

3. Update (更新)

工作流程：

- 客户端部分：

1. 使用 PRF F_{K_S} 为关键字 w 生成密钥 K_w 。
2. 获取映射 W 中当前的搜索令牌 $ST_c(w)$ 和计数器 c 。
如果不存在, 说明 w 是一个新关键字, 生成一个随机搜索令牌 $ST_0(w)$ 和计数器 $c=-1$ 。
3. 使用私钥陷门置换计算下一个搜索令牌 $ST_{c+1}(w)$:
$$ST_{c+1} = \pi_{SK}^{-1}(ST_c(w))$$
4. 更新映射 $W[w] = (ST_{c+1}(w), c + 1)$ 。
5. 计算更新令牌 UT_{c+1} :
$$UT_{c+1} = H_1(K_w, ST_{c+1})$$
6. 加密文档索引 ind :
$$e = ind \oplus H_2(K_w, ST_{c+1})$$
7. 将 (UT_{c+1}, e) 发送给服务器。

- 服务器部分：

8. 将 UT_{c+1} 和对应的密文 e 存储在映射 T 中。
-

关键组件解释

1. 前向私密性 (Forward Privacy):

- 搜索令牌 $ST_c(w)$ 与未来生成的搜索令牌 $ST_{c+1}(w)$ 无法被链接，因为只有客户端掌握私钥 SK 能进行反向计算 π^{-1} 。
- 服务器即使知道当前的搜索令牌，也无法预测未来的更新位置。

2. 安全性保证:

- H_1 和 H_2 是抗预映像的哈希函数，确保从更新令牌 UT 无法逆推出搜索令牌。
- PRF F 保证了关键字密钥 K_w 的安全性。

3. 高效性:

- 搜索过程是逆向迭代 $c \rightarrow 0$ ，计算复杂度为 $O(n_w)$ 。
- 更新过程只需要生成一个新令牌，计算复杂度较低。

正确性 (Correctness)

- **碰撞问题:** $\Sigma\phi\phi\phi\phi - B$ 的正确性依赖于更新令牌 $UT_c(w)$ 的唯一性，这由哈希函数 H_1 的抗碰撞性 (collision resistance) 来保证。
- **参数选择:** 为了确保碰撞概率可以忽略，需要选择 μ 使得 $N^2/2^\mu$ 在安全参数下为可忽略值。实际中，设置 $\mu = \lambda + 2 \log N_{\max}$ ，其中 $N_{\max}$ 是数据库支持的最大关键字-文档对数目。

复杂性 (Complexity)

- **计算复杂度:**
 - **查询 (Search):** 复杂度为 $O(n_w)$ ，与关键字 w 匹配的文档数 n_w 成正比。
 - **更新 (Update):** 复杂度为 $O(1)$ ，只需一次更新操作。
- **通信轮次:**
 - 查询和更新协议都是**单轮协议**，不会因网络延迟影响性能，与非加密协议相当。
- **带宽需求:**
 - 查询协议：每次查询仅发送一个大小为 $\log |M|$ 的搜索令牌 $ST_c(w)$ ，其中 $|M|$ 是关键字空间大小，与安全参数 λ 多项式相关。
 - 更新协议：每次更新发送一个大小为 $\mu + \ell$ 的更新令牌 $UT_c(w)$ ，比非加密数据库的最小更新令牌大小多 $\lambda + \log N_{\max}$ 位。
- **客户端存储:**
 - 客户端存储量为 $O(W(\log |M| + \log D))$ ，其中 W 是关键字数量，D 是每个关键字的最大文档数。

复杂度对比

Scheme	Computation		Communication		Client Storage	Forward Private	M.A.
	Search	Update	Search	Update			
Previous works							
[CJJ ⁺ 14]	$O(a_w)$	$O(1)$	$O(n_w)$	$O(1)$	$O(1)$	✗	✗
[BFP16]	$O(a_w + \log W)$	$O(\log W)$	$O(n_w + \log W)$	$O(\log W)$	$O(1)$	✗	✓
[SPS14]	$O\left(\min\left\{a_w + \log N, n_w \log^3 N\right\}\right)$	$O(\log^2 N)$	$O(n_w + \log N)$	$O(\log N)$	$O(N^\alpha)$	✓	✗
[SPS14] ([BFP16])	$O\left(\min\left\{a_w + \log^2 N, n_w \log^3 N\right\}\right)$	$O(\log^2 N)$	$O(n_w + \log N)$	$O(\log N)$	$O(N^\alpha)$	✓	✓ ¹
[GMP15]	$\tilde{O}(a_w \log N + \log^3 N)$	$\tilde{O}(\log^2 N)$	$\tilde{O}\left(\begin{smallmatrix} a_w \log N \\ + \log^3 N \end{smallmatrix}\right)$	$\tilde{O}(\log^3 N)$	$O(1)$	✓	✗
This work							
$\Sigma o\phi o\varsigma$	$O(a_w)$	$O(1)$	$O(n_w)$	$O(1)$	$O(W \log D)$	✓	✗
$\Sigma o\phi o\varsigma\text{-}\varepsilon$	$O(a_w)$	$O(1)$	$O(n_w)$	$O(1)$	$O(W(\log D + \lambda))$	✓	✓

Table 1 – Comparison with existing SSE schemes. N is the number of entries (*i.e.* the number of keyword/document pairs) in the database, while W is the number of distinct keywords, and D the number of documents. n_w is the size of the search result set for keyword w , and a_w is the number of times the queried keyword w was historically added to the database. In particular, in all works except [SPS14], deletions are not optimally supported: the search is not linear in the number of matching documents, but in the number of inserted documents matching the query. M.A. stands for ‘Malicious Adversary’. We omitted the polynomial dependency in the security parameter λ for both computation and communication complexity. The $\tilde{O}$ notation hides the $\log \log N$ factors. Update complexities are given per updated document/keyword pair. We only considered schemes whose server’s storage complexity is optimal ($O(N)$).

5.3 安全性

定理1 ($\Sigma\phi\phi\phi$ -B的自适应安全性) 定义 $\mathcal{L}_\Sigma = (\mathcal{L}_\Sigma^{Srch}, \mathcal{L}_\Sigma^{Updt})$ 为:

$$\begin{aligned}\mathcal{L}_\Sigma^{Srch}(w) &= (\text{sp}(w), \text{Hist}(w)) \\ \mathcal{L}_\Sigma^{Updt}(\text{add}, w, \text{ind}) &= \perp\end{aligned}$$

那么 $\Sigma\phi\phi\phi$ -B 是 $\mathcal{L}_\Sigma$ -自适应安全的

$\Sigma\phi\phi\phi - B$ 的自适应安全性基于随机预言机模型证明，通过不可区分的混合实验链构建安全性。其安全性依赖于 TDP 的单向性和 PRF 的伪随机性，且只会泄露关键词的搜索模式和插入历史，而不会泄露具体的更新操作或文档内容。这保证了协议在大部分场景下的安全性，但仍无法防御非自适应攻击。

5.4 衍生构造

支持删除

1. 删除操作的实现方法:

- 将数据结构进行**双实例化**:
 - 一个 $\Sigma\phi\phi\phi - B$ **实例**用于插入 (insertions)。
 - 另一个 $\Sigma\phi\phi\phi - B$ **实例**用于删除 (deletions)。
- 当搜索关键词 w 时，服务器分别计算两个实例中匹配 w 的文档索引集合，并返回两者的差集（即结果为插入集合减去删除集合）。

2. 泄露分析:

- 泄露信息与原方案一致，仍然是关键词的搜索模式和插入历史。
- 可以将关键词历史 $\text{HistDB}(w)$ 分为两个子列表:
 - $\text{HistDBadd}(w)$: 插入操作的历史。
 - $\text{HistDBdel}(w)$: 删除操作的历史。
- 这两个子列表分别对应插入和删除实例的泄露函数。

3. 操作隐藏优化:

- 如果在更新操作中复用相同的映射表 T ，则服务器无法区分具体的操作类型（插入或删除），从而隐藏了更新操作的具体性质。

批量更新

1. 批量更新的实现:

- 当需要更新文档索引 ind 中涉及的关键词列表 w 时:
 - 随机打乱关键词 w 的顺序。
 - 按照随机顺序依次对关键词执行原本的 **Update** 协议，输入为 ind 和对应的关键词。

2. 泄露函数不变:

- 修改后的批量更新协议不会改变泄露函数，仍然与原始方案一致。

3. 安全性证明的调整:

- 在更新过程中，模拟器需要对随机预言机 H_1, H_2 进行编程:
 - 对于关键词 w 的第 i 次更新，模拟器不再针对唯一生成的更新令牌进行编程。
 - 而是从第 i 次更新时生成的随机更新令牌中选择一个进行编程。
- 这种调整确保了批量更新的安全性证明依然成立。

5.5 减少客户端存储

原本的 $\Sigma\phi\phi\phi - B$ 需要 $O(W(\log |M| + \log D))$ 的存储，当 $|M| = Z_N^*$ （如 RSA 或 Rabin's TDP， N 为 2048 位整数）时，存储开销较大。为了解决这一问题，可以将存储降低到 $O(W \cdot \log D)$ ，代价是增加计算量。

方法如下:

1. 使用伪随机函数（PRF） G 从关键词 w 或其唯一标识符 i_w 生成初始令牌 $ST_0(w)$ ，避免直接存储令牌。
2. 当需要令牌 $ST_c(w)$ 时，通过 $ST_0(w)$ 和 c 重新计算它。这涉及迭代计算 π_{SK}^{-c} ，但对于常见 TDP（如 RSA），可以通过高效方法（如模幂运算或中国剩余定理）快速完成计算。
3. 这样，客户端只需存储 i_w 和计数器 c 。

改进后，客户端存储需求降为 $O(W \log D)$ 。这种优化后的方案被称为 $\Sigma\phi\phi\phi$ （不带 $-B$ 后缀），并可实现。其安全性证明与 $\Sigma\phi\phi\phi - B$ 类似，且适用。

5.6 防范恶意对手的安全措施

如何将 $\Sigma\phi\phi\phi$ 转变为一个可验证的 SSE 方案（称为 $\Sigma\phi\phi\phi - \varepsilon$ ），以对抗恶意攻击者:

1. 方法：客户端为每个关键词 w 存储一个 $\text{DB}(w)$ 的哈希值，并在搜索时验证服务器返回结果的哈希值是否匹配。
2. 更新：通过使用集合哈希函数实现高效的增量更新（参考 [BFP16]）。
3. 复杂度：客户端存储增加到 $O(W(\log D + \lambda))$ 。
4. 安全性：基于集合哈希函数的抗碰撞性，证明 $\Sigma\phi\phi\phi - \varepsilon$ 对恶意攻击者也是 $\mathcal{L}_\Sigma$ -自适应安全的。

结论： $\Sigma\phi\phi\phi - \varepsilon$ 通过增加**哈希验证**增强了对抗恶意攻击者的能力，同时保持较低的存储和计算开销。

5.7 与其他构造的比较

1. 功能与安全性:

如果将 $\Sigma\phi\phi\phi$ 的功能限制为只能对整篇文档进行修改（即只能添加或删除整篇文档，而不能在已有文档中增删关键词），那么它的功能与 SPS 的一致。两者的泄漏函数 L_Σ 和 L_{SPS} 虽然看起来不同，但可以互相推导。具体来说，可以通过时间戳 t 和每次更新操作中泄露的信息 $(op, ind, |w|)$ 重构出每个关键词的历史操作 $Hist(w)$ 。

由此可以证明， $\Sigma\phi\phi\phi$ 是 L_{SPS} -自适应安全的。同样，通过调整 SPS 的安全性证明，也可以证明 SPS 是 L_Σ -自适应安全的。因此， $\Sigma\phi\phi\phi$ 和 SPS 拥有完全相同的安全保证。

2. 性能比较:

- 在 **带宽和计算复杂度** 方面， $\Sigma\phi\phi\phi$ 的更新操作只需要常数级的带宽和计算复杂度，而 SPS 的更新操作需要 $O(\log^2 N)$ 的计算量，并且带宽开销为 $O(\log N)$ 。这些开销来源于 SPS 在更新中需要使用标准但较为复杂的去平摊（de-amortization）技术。
- 在 **客户端存储** 方面，SPS 需要 $O(N^\alpha)$ 的存储空间（其中 $0 < \alpha < 1$ ）来运行更新操作所需的混淆排序算法（oblivious sort），而 $\Sigma\phi\phi\phi$ 的客户端存储需求仅为 $O(W \log D)$ 。通常，关键词数 W 远小于文档数 N 。

3. 最优性:

表格 1 中的结果表明， $\Sigma\phi\phi\phi$ 在动态 SSE 方案中的渐近复杂度与最优方案相同。进一步优化前向安全方案的特性（如空间回收或存储局部性）几乎是不可能的，这表明 $\Sigma\phi\phi\phi$ 在功能和性能上的设计是最优的。这也正是其名称 $\Sigma\phi\phi\phi$ （意为“智慧”）的由来。

总结： $\Sigma\phi\phi\phi$ 在功能和安全性与 SPS 一致的情况下，在带宽利用率、更新复杂度和客户端存储需求上实现了显著优化。它的性能和设计接近理论上的最优方案。